Graph Convolutional Network

Graph Convolutional Network (GCN) adalah salah satu bentuk perkembangan dari GNN orisinal yang memanfaatkan konsep convolutional untuk menangkap informasi spektral.

- Konsep convolutional yang awalnya digunakan untuk mengekstraksi fitur spasial lokal pada data grid (seperti citra dalam CNN), kemudian digeneralisasi agar dapat beroperasi pada data graf.
- Dengan demikian, GCN mampu melakukan proses agregasi informasi dari node tetangga secara iteratif, sehingga representasi setiap node tidak hanya mencerminkan fitur dirinya sendiri, tetapi juga struktur dan informasi di sekitarnya.

Alur GCN

Feature setiap node

↓

Ambil informasi diri sendiri + tetangga

↓

Normalisasi kontribusi tetangga

↓

Aggregate

↓

Transform dengan shared learnable weight *W*

↓

Tambah shared bias b

↓

Fungsi Aktivasi (ReLU)

↓

Feature node yang baru

Forward Propagation

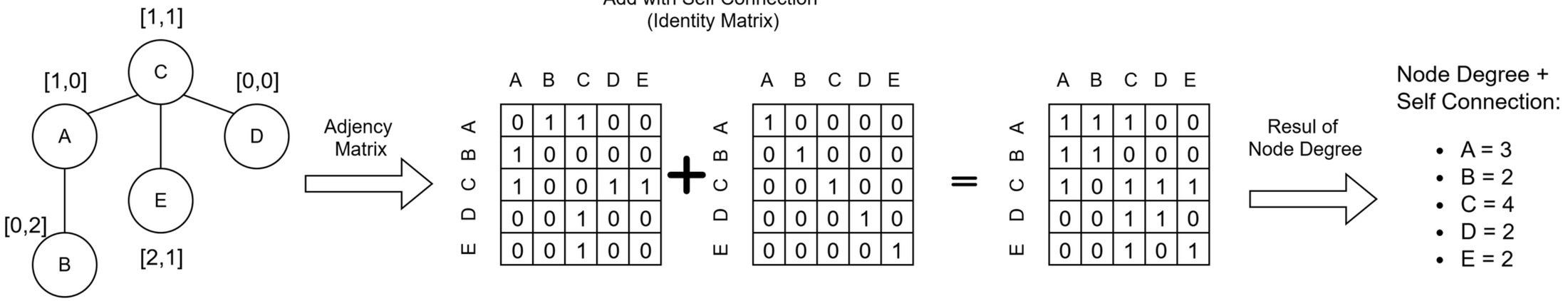
Cara Kerja GCN:

- Sama seperti dengan GNN, message passing dari seluruh tetangga diterima terlebih dahulu. Kemudian, menghitung degree dari masing-masing node (Adjency Matrix), dengan ditambah 1 (self connection) (Identity Matrix).
  - Hal ini berfungsi untuk memperhatikan kepentingan dari representasi diri *node* itu sendiri

$$\tilde{A} = A + I_N$$

Add with Self Connection
(Identity Matrix)

$$\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$$

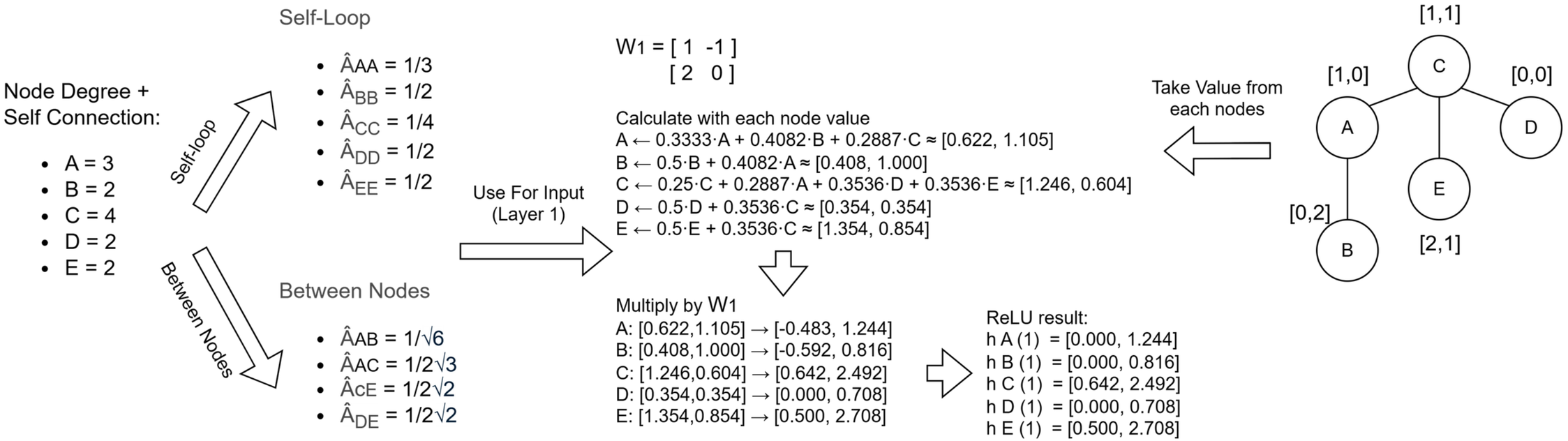


Forward Propagation (Cont...)

Cara Kerja GCN:

- Selanjutnya, menghitung nilai ***degree normalization factor*** untuk masing-masing *node*.
- Hasil dari *degree normalization factor*, kita dapat menggunakan nilai tersebut untuk menyatukan message passing dari *node* tetangga dan dapat diteruskan ke fungsi aktivasi

$$\hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \qquad H^{(1)} = \mathrm{ReLU}(\hat{A} H^{(0)} W_1)$$



Backpropagation and Learning

Proses *backpropagation* sama dengan MLP, dimana hasil perhitungan dibandingkan dengan nilai riil. Kemudian, dihitung untuk nilai loss untuk memulai backpropagation.

Hitung Loss dari hasil prediksi

↓

Hitung gradien untuk shared learnable weight *W* dan shared bias *b*

↓

Gunakan optimizer

↓

Update *W* dan *b*

↓

Lakukan kembali forward propagation